

Replace Temp with Query

تعریف ساده

متغیرهای موقتی (temp variables) که نتیجه یه محاسبه رو نگه میدارن رو حذف کن و به جاش یه متد بساز که همون مقدار رو برگردونه.

چرا این کار رو می‌کنیم؟

متغیرهای موقتی (اونایی که توی یه متد تعریف میشن و موقتاً یه مقدار رو نگه میدارن) چندتا مشکل ایجاد می‌کنن:

مشکل	توضیح
طولانی شدن متد	هر متغیر موقتی چند خط به متد اضافه می‌کنه
مانع Extract Method	وقتی می‌خوای بخشی از کد رو استخراج کنی، این متغیرها محلی هستن و دست و پا گیر میشن
تکرار محاسبات	ممکنه چندجا نیاز بشه همون محاسبه انجام بشه
کمک به Long Method	وجود تعداد زیادی متغیر موقتی، نشونه Long Method هست

مثال ساده (قبل از ریفکتور)

فرض کن این کد رو داری:

csharp

```
public decimal CalculateFinalPrice(Product product)
{
    // متغیرهای موقتی
    ;decimal basePrice = product.Price * product.Quantity
    ;decimal discount = 0

    if (product.IsOnSale)
```

```

    }

    ;discount = basePrice * 0.1m
    {
        else if (product.IsSeasonal)
    }
    ;discount = basePrice * 0.05m
    {

        // مالیات ۹٪ ;decimal tax = basePrice * 0.09m
        ;decimal finalPrice = basePrice - discount + tax

        ;return finalPrice
    }

```

اینجا ۴ تا متغیر موقتی داریم: basePrice , discount , tax , finalPrice

بعد از ریفکتور 

csharp

```

public decimal CalculateFinalPrice(Product product)
{
    ;decimal basePrice = GetBasePrice(product)
    ;decimal discount = GetDiscount(product, basePrice)
    ;decimal tax = GetTax(basePrice)

    ;return basePrice - discount + tax
}

private decimal GetBasePrice(Product product)
{
    ;return product.Price * product.Quantity
}

private decimal GetDiscount(Product product, decimal basePrice)
{
    if (product.IsOnSale)
        ;return basePrice * 0.1m

    if (product.IsSeasonal)
        ;return basePrice * 0.05m
}

```

```

        ;return 0
    }

    private decimal GetTax(decimal basePrice)
    {
        ;return basePrice * 0.09m
    }

```



مزایا

مزیت

توضیح

متد اصلی خواناتر	CalculateFinalPrice فقط جریان کار رو نشون میده
قابلیت استفاده مجدد	GetTax و GetDiscount رو میتونم جاهای دیگه هم صدا بزنم
تست آسون تر	میتونم GetDiscount رو جداگونه تست کنم
کمک به Extract Method	حالا دیگه متغیرهای محلی مانع شکستن متد نیستن

تمرین برای تو

این کد رو با **Replace Temp with Query** ریفتکتور کن:

```

csharp

public class OrderProcessor
{
    public decimal ProcessOrder(Order order)
    {
        // محاسبه قیمت پایه
        ;decimal itemsTotal = 0
        foreach (var item in order.Items)
        {
            ;itemsTotal += item.Price * item.Quantity
        }
    }
}

```

```

    {

        // محاسبه وزن کل
        ;decimal totalWeight = 0
        foreach (var item in order.Items)
        }

        ;totalWeight += item.Weight * item.Quantity

    {

        // محاسبه هزینه حمل
        ;decimal shippingCost = 0
        if (totalWeight < 5)
        ;shippingCost = 10000
        else if (totalWeight < 20)
        ;shippingCost = 25000
        else
        ;shippingCost = 50000

        // محاسبه تخفیف
        ;decimal discount = 0
        if (itemsTotal > 500000)
        ;discount = itemsTotal * 0.15m
        else if (itemsTotal > 200000)
        ;discount = itemsTotal * 0.05m

        // محاسبه مالیات
        ;decimal tax = (itemsTotal - discount) * 0.09m

        // قیمت نهایی
        ;decimal finalPrice = itemsTotal - discount + tax + shippingCost

        ;return finalPrice
    }
}

```

راهنمایی:

- itemsTotal → میتونه بشه CalculateItemsTotal()
- totalWeight → میتونه بشه CalculateTotalWeight()
- shippingCost → میتونه بشه CalculateShippingCost(totalWeight)
- discount → میتونه بشه CalculateDiscount(itemsTotal)

• tax → میتونه بشه CalculateTax(itemsTotal, discount)

کدت رو بفرست تا باهم بررسی کنیم! 🙌

کلاس کمکیشم بفرست تا سریع تر کار بره جلو

چشم، بفرما:

csharp

```
public class Order
{
    public List<OrderItem> Items { get; set; } = new List<OrderItem>
    {

        public class OrderItem
        {
            public decimal Price { get; set; }
            public int Quantity { get; set; }
            public decimal Weight { get; set; }
        }
    }
}
```

منتظر کد ریفتور شده‌ات هستم! 🖥️

```
public class OrderProcessor_After
{
    private decimal CalculateItemsTotal(Order order)
    {
        decimal itemsTotal = 0
        foreach (var item in order.Items)
        {
            itemsTotal += item.Price * item.Quantity
        }
        return itemsTotal
    }

    private decimal CalculateTotalWeight(Order order)
    {
        decimal totalWeight = 0
        foreach (var item in order.Items)
        {
            totalWeight += item.Weight * item.Quantity
        }
    }
}
```